



main.py



Share

Run



JS

TS

```
1 arr = [10, 25, 30, 45, 50]
2 key = 30
3
4 found = False
5
6 for i in range(len(arr)):
7     if arr[i] == key:
8         print("Key found at index", i)
9         found = True
10        break
11
12 if not found:
13     print("Key not found")
14
15
16
17 def binary_search(arr, low, high, key):
18     if low <= high:
19         mid = (low + high) // 2
20
21         if arr[mid] == key:
22             return mid
23         elif key < arr[mid]:
```

Output

Clear

```
Key found at index 2
Key found at index 3
Iterative: 120
Recursive: 120
0 1 1 2 3 5 [[19, 22], [43, 50]]
[3, 9, 27, 38, 43]
[1, 5, 7, 8, 9, 10]
[11, 12, 22, 25, 34, 64, 90]
[11, 12, 22, 25, 64]
[5, 6, 11, 12, 13]
[1, 3, 4, 5, 10]
Inversions = 3
Maximum Sum = 7
1024
Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979
```

=== Code Execution Successful ===

main.py	Share Run	Output Clear
	<pre>24 return binary_search(arr, low, mid - 1, key) 25 else: 26 return binary_search(arr, mid + 1, high, key) 27 return -1 28 29 arr = [5, 10, 15, 20, 25] 30 key = 20 31 32 result = binary_search(arr, 0, len(arr)-1, key) 33 34 if result != -1: 35 print("Key found at index", result) 36 else: 37 print("Key not found") 38 39 40 41 def factorial_recursive(n): 42 if n == 0 or n == 1: 43 return 1 44 return n * factorial_recursive(n-1)</pre>	<pre>Key found at index 2 Key found at index 3 Iterative: 120 Recursive: 120 0 1 1 2 3 5 [[19, 22], [43, 50]] [3, 9, 27, 38, 43] [1, 5, 7, 8, 9, 10] [11, 12, 22, 25, 34, 64, 90] [11, 12, 22, 25, 64] [5, 6, 11, 12, 13] [1, 3, 4, 5, 10] Inversions = 3 Maximum Sum = 7 1024 Closest Pair: ((1, 2), (3, 1)) Distance: 2.23606797749979 === Code Execution Successful ===</pre>



main.py



Run



JS

TS

```
46 n = 5
47
48 fact = 1
49 for i in range(1, n+1):
50     fact *= i
51
52 print("Iterative:", fact)
53 print("Recursive:", factorial_recursive(n))
54
55
56 n = 6
57
58 a = 0
59 b = 1
60
61 print(a, end=" ")
62 print(b, end=" ")
63
64 for i in range(2, n):
65     c = a + b
66     print(c, end=" ")
67     a = b
68     b = c
```

Output

Clear

```
Key found at index 2
Key found at index 3
Iterative: 120
Recursive: 120
0 1 1 2 3 5 [[19, 22], [43, 50]]
[3, 9, 27, 38, 43]
[1, 5, 7, 8, 9, 10]
[11, 12, 22, 25, 34, 64, 90]
[11, 12, 22, 25, 64]
[5, 6, 11, 12, 13]
[1, 3, 4, 5, 10]
Inversions = 3
Maximum Sum = 7
1024
Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979
```

```
=== Code Execution Successful ===
```



main.py



Run

Output

Clear

```
68     b = c
69
70
71 A = [[1,2],[3,4]]
72 B = [[5,6],[7,8]]
73
74 result = [[0,0],[0,0]]
75
76 for i in range(2):
77     for j in range(2):
78         for k in range(2):
79             result[i][j] += A[i][k] * B[k][j]
80
81 print(result)
82
83
84 def merge_sort(arr):
85     if len(arr) > 1:
86         mid = len(arr)//2
87
88         left = arr[:mid]
89         right = arr[mid:]
90
```

```
Key found at index 2
Key found at index 3
Iterative: 120
Recursive: 120
0 1 1 2 3 5 [[19, 22], [43, 50]]
[3, 9, 27, 38, 43]
[1, 5, 7, 8, 9, 10]
[11, 12, 22, 25, 34, 64, 90]
[11, 12, 22, 25, 64]
[5, 6, 11, 12, 13]
[1, 3, 4, 5, 10]
Inversions = 3
Maximum Sum = 7
1024
Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979
```

=== Code Execution Successful ===



JS

TS

main.py



Run

```
91 merge_sort(left)
92 merge_sort(right)
93
94 i = j = k = 0
95
96 while i < len(left) and j < len(right):
97     if left[i] < right[j]:
98         arr[k] = left[i]
99         i += 1
100     else:
101         arr[k] = right[j]
102         j += 1
103     k += 1
104
105 while i < len(left):
106     arr[k] = left[i]
107     i += 1
108     k += 1
109
110 while j < len(right):
111     arr[k] = right[j]
112     j += 1
113     k += 1
```

Output

Clear

Key found at index 2
Key found at index 3
Iterative: 120
Recursive: 120
0 1 1 2 3 5 [[19, 22], [43, 50]]
[3, 9, 27, 38, 43]
[1, 5, 7, 8, 9, 10]
[11, 12, 22, 25, 34, 64, 90]
[11, 12, 22, 25, 64]
[5, 6, 11, 12, 13]
[1, 3, 4, 5, 10]
Inversions = 3
Maximum Sum = 7
1024
Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979

=== Code Execution Successful ===



main.py

 Share

Run

```
115 arr = [38,27,43,3,9]
116 merge_sort(arr)
117
118 print(arr)
119
120
121
122 def quick_sort(arr):
123     if len(arr) <= 1:
124         return arr
125
126     pivot = arr[0]
127
128     left = [x for x in arr[1:] if x <= pivot]
129     right = [x for x in arr[1:] if x > pivot]
130
131     return quick_sort(left) + [pivot] + quick_sort(right)
132
133 arr = [10,7,8,9,1,5]
134
135 print(quick_sort(arr))
136
137 arr = [64.34.25.12.22.11.90]
```

Output

Key found at index 2
Key found at index 3
Iterative: 120
Recursive: 120
0 1 1 2 3 5 [[19, 22], [43, 50]]
[3, 9, 27, 38, 43]
[1, 5, 7, 8, 9, 10]
[11, 12, 22, 25, 34, 64, 90]
[11, 12, 22, 25, 64]
[5, 6, 11, 12, 13]
[1, 3, 4, 5, 10]
Inversions = 3
Maximum Sum = 7
1024
Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979

=== Code Execution Successful ===



12 Samples
(less of spring flowers and leaves)

Shape it. Fill it. Love it.

The new Generative Shape Fill.

Try now

Adobe

Illustrator



main.py

 Share

Run

```
139 n = len(arr)
140
141 for i in range(n):
142     for j in range(0, n-i-1):
143         if arr[j] > arr[j+1]:
144             arr[j], arr[j+1] = arr[j+1], arr[j]
145
146 print(arr)
147
148
149 arr = [64,25,12,22,11]
150
151 for i in range(len(arr)):
152     minimum = i
153
154     for j in range(i+1, len(arr)):
155         if arr[j] < arr[minimum]:
156             minimum = j
157
158     arr[i], arr[minimum] = arr[minimum], arr[i]
159
160 print(arr)
```

Output

Clear

Key found at index 2
Key found at index 3
Iterative: 120
Recursive: 120
0 1 1 2 3 5 [[19, 22], [43, 50]]
[3, 9, 27, 38, 43]
[1, 5, 7, 8, 9, 10]
[11, 12, 22, 25, 34, 64, 90]
[11, 12, 22, 25, 64]
[5, 6, 11, 12, 13]
[1, 3, 4, 5, 10]
Inversions = 3
Maximum Sum = 7
1024
Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979

=== Code Execution Successful ===



main.py

 Share

 Run

```
139 n = len(arr)
140
141 for i in range(n):
142     for j in range(0, n-i-1):
143         if arr[j] > arr[j+1]:
144             arr[j], arr[j+1] = arr[j+1], arr[j]
145
146 print(arr)
147
148
149 arr = [64,25,12,22,11]
150
151 for i in range(len(arr)):
152     minimum = i
153
154     for j in range(i+1, len(arr)):
155         if arr[j] < arr[minimum]:
156             minimum = j
157
158     arr[i], arr[minimum] = arr[minimum], arr[i]
159
160 print(arr)
```

Output

Clear

Key found at index 2
Key found at index 3
Iterative: 120
Recursive: 120
0 1 1 2 3 5 [[19, 22], [43, 50]]
[3, 9, 27, 38, 43]
[1, 5, 7, 8, 9, 10]
[11, 12, 22, 25, 34, 64, 90]
[11, 12, 22, 25, 64]
[5, 6, 11, 12, 13]
[1, 3, 4, 5, 10]
Inversions = 3
Maximum Sum = 7
1024
Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979

=== Code Execution Successful ===



JS

TS

main.py

 Share

Run

```
188 print(sorted_arr)
189
190
191
192 count = 0
193
194 arr = [2,4,1,3,5]
195
196 for i in range(len(arr)):
197     for j in range(i+1, len(arr)):
198         if arr[i] > arr[j]:
199             count += 1
200
201 print("Inversions =", count)
202
203
204
205 arr = [-2,-3,4,-1,-2,1,5,-3]
206
207 max_sum = arr[0]
208 current_sum = arr[0]
209
210 for i in range(1, len(arr)):
```

Output

Key found at index 2
Key found at index 3
Iterative: 120
Recursive: 120
0 1 1 2 3 5 [[19, 22], [43, 50]]
[3, 9, 27, 38, 43]
[1, 5, 7, 8, 9, 10]
[11, 12, 22, 25, 34, 64, 90]
[11, 12, 22, 25, 64]
[5, 6, 11, 12, 13]
[1, 3, 4, 5, 10]
Inversions = 3
Maximum Sum = 7
1024
Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979

=== Code Execution Successful ===



main.py



Share

Run

Output

```
221
222 print(x ** n)
223
224
225
226 import math
227
228 points = [(1,2), (4,5), (7,8), (3,1)]
229
230 minimum = float('inf')
231
232 pair = ()
233
234 for i in range(len(points)):
235     for j in range(i+1, len(points)):
236         d = math.dist(points[i], points[j])
237
238         if d < minimum:
239             minimum = d
240             pair = (points[i], points[j])
241
242 print("Closest Pair:", pair)
243 print("Distance:", minimum)
```

```
Key found at index 2
Key found at index 3
Iterative: 120
Recursive: 120
0 1 1 2 3 5 [[19, 22], [43, 50]]
[3, 9, 27, 38, 43]
[1, 5, 7, 8, 9, 10]
[11, 12, 22, 25, 34, 64, 90]
[11, 12, 22, 25, 64]
[5, 6, 11, 12, 13]
[1, 3, 4, 5, 10]
Inversions = 3
Maximum Sum = 7
1024
Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979
```

=== Code Execution Successful ===

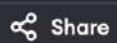


JS

TS



main.py



Run

Output

Clear

```
1 def merge_count(arr, temp, left, mid, right):
2     i, j, k = left, mid + 1, left
3     inv_count = 0
4
5     while i <= mid and j <= right:
6         if arr[i] <= arr[j]:
7             temp[k] = arr[i]
8             i += 1
9         else:
10            temp[k] = arr[j]
11            inv_count += (mid - i + 1) # all remaining elements in left half
                                     # are inversions
12            j += 1
13        k += 1
14
15    while i <= mid:
16        temp[k] = arr[i]
17        i += 1
18        k += 1
19    while j <= right:
20        temp[k] = arr[j]
21        j += 1
22        k += 1
23
24    for idx in range(left, right + 1):
25        arr[idx] = temp[idx]
```

Number of inversions: 3

=== Code Execution Successful ===



main.py



Run

Output

Clear

```
1 def kadane(arr):
2     max_so_far = arr[0]
3     max_ending_here = arr[0]
4     for i in range(1, len(arr)):
5         max_ending_here = max(arr[i], max_ending_here + arr[i])
6         max_so_far = max(max_so_far, max_ending_here)
7     return max_so_far
8
9
10 def max_crossing_sum(arr, left, mid, right):
11     left_sum = float('-inf')
12     total = 0
13     for i in range(mid, left - 1, -1):
14         total += arr[i]
15         left_sum = max(left_sum, total)
16
17     right_sum = float('-inf')
18     total = 0
19     for i in range(mid + 1, right + 1):
20         total += arr[i]
21         right_sum = max(right_sum, total)
22
23     return left_sum + right_sum
24
25
26 def max_subarray_dc(arr, left, right):
27     if left == right:
28         return arr[left]
```

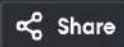
Kadane's Algorithm Result: 7

Divide and Conquer Result: 7

=== Code Execution Successful ===



main.py



Run

Output

Clear

```
1 def power_iterative(x, n):
2     result = 1
3     for _ in range(n):
4         result *= x
5     return result
6
7
8 def power_recursive(x, n):
9     if n == 0:
10        return 1
11    if n % 2 == 0:
12        half = power_recursive(x, n // 2)
13        return half * half
14    else:
15        half = power_recursive(x, n // 2)
16        return half * half * x
17
18
19 if __name__ == "__main__":
20     x, n = 2, 10
21     print("Iterative Power:", power_iterative(x, n))
22     print("Recursive Fast Power:", power_recursive(x, n))
```

Iterative Power: 1024

Recursive Fast Power: 1024

=== Code Execution Successful ===



main.py



Share

Run

Output

Clear

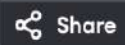
```
1 import math
2
3
4 def distance(p1, p2):
5     return math.sqrt((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2)
6
7
8 def brute_force(points):
9     min_dist = float('inf')
10    pair = None
11    n = len(points)
12    for i in range(n):
13        for j in range(i + 1, n):
14            d = distance(points[i], points[j])
15            if d < min_dist:
16                min_dist = d
17                pair = (points[i], points[j])
18    return min_dist, pair
19
20
21 def closest_pair_rec(px, py):
22     n = len(px)
23     if n <= 3:
24         return brute_force(px)
25
26     mid = n // 2
```

```
Brute Force -> Closest pair: ((1, 2), (3, 1)), Distance: 2.2361
Divide & Conquer -> Closest pair: ((1, 2), (3, 1)), Distance: 2.2361

=== Code Execution Successful ===
```



main.py



Run



```
1 import math
2
3
4 def distance(p1, p2):
5     return math.sqrt((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2)
6
7
8 def brute_force(points):
9     min_dist = float('inf')
10    pair = None
11    n = len(points)
12    for i in range(n):
13        for j in range(i + 1, n):
14            d = distance(points[i], points[j])
15            if d < min_dist:
16                min_dist = d
17                pair = (points[i], points[j])
18    return min_dist, pair
19
20
21 def closest_pair_rec(px, py):
22     n = len(px)
23     if n <= 3:
24         return brute_force(px)
25
26     mid = n // 2
```

Output

Clear

Brute Force -> Closest pair: ((1, 2), (3, 1)), Distance: 2.2361
Divide & Conquer -> Closest pair: ((1, 2), (3, 1)), Distance: 2.2361

=== Code Execution Successful ===